

Phase2-kaushal Production Readiness Report

Date: 2026-07-13

Branch checked: 'Phase2-kaushal'

Remote: 'origin/Phase2-kaushal'

Latest checked commit: '079aaf6 feat: implement comprehensive HRMS core modules including attendance, leave, and master data services with full-stack API integration.'

1. Branch Status

- 'Phase2-kaushal' was already checked out locally.
- 'git fetch origin --prune' completed successfully.
- 'git pull --ff-only origin Phase2-kaushal' returned 'Already up to date'.
- 'Phase1-kaushal' is already included in 'Phase2-kaushal'.
- Verification command: 'git merge-base --is-ancestor origin/Phase1-kaushal HEAD'
- Result: exit code '0', which means Phase 1 history is present in Phase 2.

2. Phase 1 Coverage Check

Phase 1 expected flow from the document:

- Admin login
- Admin dashboard
- Admin creates HR
- Admin views HR and employee list
- HR login
- HR dashboard
- HR creates employee with login access
- HR employee list/detail/edit
- HR attendance monitoring
- Employee login
- Employee dashboard
- Employee punch in/out
- Employee attendance history
- Employee profile
- Employee change password

Implementation status:

- Backend routes exist for auth, HR users, employees, attendance, dashboard, profile, notifications, login activity.
- Frontend routes exist for admin/master dashboard, HR dashboard, employee dashboard, employees, attendance, profile, change password, login activity.
- Phase 1 code is included in the Phase 2 branch.
- Functional build and automated tests pass after dependency installation and dependency hardening.

Phase 1 verdict:

- Phase 1 is functionally present in 'Phase2-kaushal'.
- No extra Phase 1 merge was required.

3. Phase 2 Coverage Check

Phase 2 expected scope from the document:

- Time off module
- Approval center APIs
- Attendance regularization workflow
- Master data CRUD APIs

- Report and export APIs
- Stronger notification integration
- Refined login activity and security flow
- Server-side filtering and pagination

Implementation status:

- Backend includes 'timeoff_routes.py'.
- Backend includes 'approval_routes.py'.
- Backend includes 'regularization_routes.py'.
- Backend includes 'master_data_routes.py'.
- Backend includes 'report_routes.py'.
- Backend includes notification and login activity routes.
- Frontend includes time off pages for employee and HR/master usage.
- Frontend includes regularization pages for employee and HR/master review.
- Frontend includes master data page.
- Frontend includes reports pages for HR and master/admin.
- Frontend includes navbar notification handling and WebSocket wiring.
- Employee, attendance, time off, and regularization APIs include pagination/filtering in multiple places.

Phase 2 verdict:

- Phase 2 structure and main modules are implemented.
- Automated tests pass.
- Remaining items are release-hardening and final server smoke checks, not missing core module scaffolding.

4. Fixes Applied During Audit

These fixes were made without changing UI screens:

- Added missing backend test dependency: 'httpx2==2.5.0' in 'backend/requirements.txt'.
- Updated Angular package ranges in 'frontend/package.json' to patched Angular 21 versions.
- Regenerated 'frontend/package-lock.json'.
- Reduced npm audit result from '32 vulnerabilities, 14 high' to '1 low severity vulnerability'.

Files changed:

- 'backend/requirements.txt'
- 'frontend/package.json'
- 'frontend/package-lock.json'

5. Verification Commands And Results

Frontend:

- Command: 'npm run build'
- Result: passed
- Output folder: 'frontend/dist/frontend'
- Command: 'npm test -- --watch=false'
- Result: passed
- Test result: '16 passed', '17 tests passed'
- Command: 'npm audit --audit-level=high'
- Result: passed for high severity
- Remaining audit item: '1 low severity vulnerability' in 'esbuild'

Backend:

- Command: 'DATABASE_URL='sqlite:///tmp/aivan_hrms_audit.db' JWT_SECRET_KEY='audit-secret'

```
python3 -m compileall app'
- Result: passed

- Command: 'DATABASE_URL='sqlite:////tmp/aivan_hrms_pytest.db' JWT_SECRET_KEY='audit-secret'
python -m pytest -q'
- Result: passed
- Test result: '40 passed'

- Command: 'DATABASE_URL='sqlite:////tmp/aivan_hrms_startup.db' JWT_SECRET_KEY='audit-secret'
APP_ENV='production' AUTO_CREATE_TABLES='false' AUTO_SEED_ROLES='false'
AUTO_SEED_DEMO_DATA='false' ENABLE_SCHEDULER='false' EXPOSE_RESET_LINK_IN_RESPONSE='false'
python -c 'from app.main import app; print(app.title)''
- Result: passed
- Output: 'HRMS API'

- Command: 'alembic heads'
- Result: passed
- Single migration head: '9c76cbfeaf96'
```

6. Remaining Fixes Before Final Production Deployment

P1 - Run Alembic migration smoke test on real PostgreSQL

Status:

- Local SQLite migration smoke failed because SQLite does not support the 'drop_constraint' operation used in one migration.
- This is a SQLite limitation, not proof that PostgreSQL will fail.
- Production target is PostgreSQL, so the final check must be done on the real server DB.

Action:

- On server/staging PostgreSQL DB, run:

```
'''powershell
cd C:\apps\Aivan-HRMS-Portal\backend
venv\Scripts\activate
alembic upgrade head
'''
```

Acceptance:

- Migration completes with no error.
- 'alembic current' shows '9c76cbfeaf96'.

P1 - Configure production environment variables on server

Status:

- Backend fails fast if 'DATABASE_URL' or 'JWT_SECRET_KEY' is missing.
- This is good for production, but server setup must be correct.

Action:

- Create backend '.env' on the Windows server with production values.

Required values:

```
'''env
APP_ENV=production
DATABASE_URL=postgresql+psycopg://USER:PASSWORD@HOST:5432/DB_NAME
JWT_SECRET_KEY=use-a-long-random-secret
JWT_ALGORITHM=HS256
```

```
BACKEND_CORS_ORIGINS=http://YOUR_DOMAIN_OR_SERVER_IP
FRONTEND_URL=http://YOUR_DOMAIN_OR_SERVER_IP
AUTO_CREATE_TABLES=false
AUTO_SEED_ROLES=false
AUTO_SEED_DEMO_DATA=false
ENABLE_SCHEDULER=false
EXPOSE_RESET_LINK_IN_RESPONSE=false
'''
```

Acceptance:

- 'uvicorn app.main:app --host 127.0.0.1 --port 8000' starts successfully.
- 'http://127.0.0.1:8000/health' returns status 'ok'.

P1 - Create or confirm first production admin user

Status:

- Production flags should keep demo data disabled.
- If demo data is disabled on a fresh DB, first real admin user must still exist.

Action:

- Either create the first admin through a controlled one-time seed script.
- Or temporarily seed demo data only in staging, then replace demo credentials before public release.
- Do not leave default demo credentials active on a public server.

Acceptance:

- A real admin can log in.
- Demo credentials are removed, disabled, or changed before release.

P1 - IIS reverse proxy must support API and WebSocket traffic

Status:

- Frontend production API base is '/api/v1'.
- WebSocket base resolves from current host.
- This is correct for a same-domain IIS deployment, but IIS must forward both HTTP and WebSocket traffic to FastAPI.

Action:

- Serve Angular static files from IIS.
- Reverse proxy '/api/*' to 'http://127.0.0.1:8000/api/*'.
- Reverse proxy '/ws/*' to 'ws://127.0.0.1:8000/ws/*'.
- Enable WebSocket Protocol in Windows/IIS if notifications are required.

Acceptance:

- Browser can load frontend.
- Login API works from browser.
- Notifications/WebSocket connection does not fail in browser dev tools.

P2 - Remove or gate production console logs

Status:

- Multiple frontend 'console.log' debug statements remain in employee services, employee modals, dashboard profile handlers, and WebSocket connect logs.
- These do not block deployment, but they are not polished for production.

Action:

- Replace debug logs with environment-gated logging or remove them.
- Keep 'console.error' only where useful for real error diagnosis.

Acceptance:

- Browser console does not show routine debug logs during normal HR/employee flows.

P2 - Clean backend deprecation warnings

Status:

- Tests pass, but warnings exist for Pydantic v2 deprecated class-based 'Config', 'json_encoders', and FastAPI 'on_event'.
- These are not current deploy blockers.

Action:

- Move schemas to 'ConfigDict'.
- Replace old 'json_encoders' with Pydantic v2 serializers where needed.
- Replace FastAPI '@app.on_event' startup/shutdown with lifespan handlers.

Acceptance:

- 'pytest -q' passes with materially fewer deprecation warnings.

P3 - Remaining low severity npm audit item

Status:

- 'npm audit --audit-level=high' passes.
- 'npm audit --audit-level=moderate' also passes for moderate/high.
- One low severity 'esbuild' advisory remains.
- Advisory applies to development server file access behavior, not the generated production static bundle.

Action:

- Track and resolve when Angular build chain publishes a compatible patched dependency.
- Do not use 'ng serve' as the production server.

Acceptance:

- Production uses IIS/static build only.
- Future dependency update clears 'npm audit'.

7. Deployment Readiness Verdict

Current status:

- Code is much closer to production-ready after dependency hardening.
- Phase 1 code is present in Phase 2.
- Phase 2 modules are present.
- Frontend production build passes.
- Frontend tests pass.
- Backend tests pass.
- Backend app imports successfully with production-style flags.
- High severity npm audit findings are cleared.

Release verdict:

- Ready for staging deployment.

- Ready for production deployment after the P1 server checks above are completed on the actual Windows/PostgreSQL server.

Do not publish publicly until:

- PostgreSQL 'alembic upgrade head' passes.
- Real admin login is confirmed.
- IIS '/api' and '/ws' proxy is confirmed.
- Demo credentials are not active on public production.